

Crescendo



A web-based platform that listens to music, detects notes & chords, and provides interactive tools for musicians and learners.

Praggo Protim Datta

Roll: 2305060

Fahmid Hisham

Roll: 2305058

Understanding Music

What are Notes and Chords?



What is a Musical Note?

- A **note** is a single sound at a specific **pitch**
- Pitch is determined by the **frequency** of vibration
- Western music uses **12 notes** that repeat in **octaves**

The 12 Notes

C C# D D# E F F# G G# A
A# B

Then it repeats in the next octave

Note	Frequency
C4	261.6 Hz
E4	329.6 Hz
G4	392.0 Hz
A4	440.0 Hz
C5	523.3 Hz

A4 = 440 Hz is the universal tuning standard



What is a Chord?

- A **chord** is multiple notes played **simultaneously**
- Built by stacking notes at specific **intervals**
- Creates the **harmony** and emotional feel of music

Example: C Major

C (root) + **E** (3rd) + **G** (5th)

Intervals: 4 semitones, then 3 semitones

Common Chord Types

- **Major** — bright, happy
- **Minor** — dark, sad
- **Diminished** — tense
- **7th** — jazzy, rich
- **Sus** — unresolved

A chord's *quality* depends entirely on the *intervals* between its notes.



Notes to Chords — Visual Example



Played together

C Major

C + E + G

Bright & Happy

A Minor

A + C + E

Sad & Soft

G7

G + B + D + F

Jazzy & Tense

Resulting Chords:

The Problem

Humans hear music. Computers see numbers.



Just Detect Frequencies? Not So Simple!

A single note contains many frequencies — its **fundamental** plus its **harmonics**.

One note “C3” produces:

C3 fundamental	130.8 Hz
2nd harmonic	261.6 Hz
3rd harmonic	392.4 Hz
4th harmonic	523.2 Hz
5th harmonic	654.0 Hz
:	

What a naive detector sees:

130 Hz	→	C3
262 Hz	→	C4
392 Hz	→	G4
523 Hz	→	C5
654 Hz	→	E5

Wrong! It thinks 5 notes were played,
but really only **one** note (C3) was pressed.

Detecting frequencies is not enough — we must **separate real notes from harmonics**. This is why we need CQT, NMF, ghost filtering, and music theory.

DSP Systems

Translating Sound into Music

CQT — Constant-Q Transform

- Converts audio from the **time domain** to the **frequency domain**
- Uses **logarithmic frequency bins** aligned with musical notes
- Each bin corresponds to one **semitone**

Why CQT instead of FFT?

- Bins match piano keys exactly
- Better low-frequency resolution
- Octave-invariant structure
- Suited for polyphonic detection

CQT Formula

$$X_k = \sum_{n=0}^{N_k-1} x[n] w_k[n] e^{-j2\pi Qn/N_k}$$

Q = constant quality factor

N_k = window length per bin

$$f_k = f_{\min} \cdot 2^{k/b}$$

b = bins per octave



NMF — Non-Negative Matrix Factorization

- Decomposes the CQT spectrum into **templates** and **activations**
- W** = what each note looks like in frequency
- H** = when and how loud each note plays



Decomposition

$$\mathbf{V} \approx \mathbf{W} \cdot \mathbf{H}$$

V = CQT magnitude spectrogram (freq bins $\times$ time)

W = note recipes (columns = notes, rows = freq bins)

H = note activations (rows = notes, columns = time)

Advantages

- Separates overlapping notes
- Learns harmonic patterns
- Handles polyphonic audio



NMF Example — What Do **W** and **H** Store?

Example with $F = 4$ frequency bins, $N = 3$ possible notes, and $T = 4$ time frames

W ($F \times N$) — Note “Flavors”

	C4	E4	G4
f_1	1.0	0.1	0.0
f_2	0.4	1.0	0.2
f_3	0.2	0.3	1.0
f_4	0.1	0.1	0.4

H ($N \times T$) — Note Activity

	t_1	t_2	t_3	t_4
C4	1.0	0.8	0.0	0.2
E4	0.7	0.6	0.0	0.1
G4	0.9	0.7	0.1	0.0

How the multiplication works

W: rows = frequency bins; columns = note fingerprints (one column per detectable note).

H: rows = the same notes; columns = time frames; $H_{n,t}$ is note n 's strength at time t .

$\underbrace{\mathbf{W}_{:,n}}_{F \times 1} \underbrace{\mathbf{H}_{n,:}}_{1 \times T} = \underbrace{\mathbf{V}^{(n)}}_{F \times T}$: an outer product giving note n 's contribution across frequency and time.

$$\mathbf{V} \approx \mathbf{WH} = \sum_{n=1}^N \mathbf{W}_{:,n} \mathbf{H}_{n,:}, \quad V_{f,t} = \sum_{n=1}^N W_{f,n} H_{n,t}$$

At t_1 : C4 = 1.0, E4 = 0.7, G4 = 0.9; three strong templates indicate a likely C-major chord.



Methods Summary — Problem & Solution

Problem	Method	What it Does
Raw waveform hard to analyze	STFT / CQT	Convert sound into frequency information
Music is logarithmic	CQT	Frequency bins follow musical pitch
One note has many harmonics	Harmonic Templates	Learn the fingerprint of each note
Piano strings aren't ideal	Fletcher Model	Model real piano harmonics
Multiple notes at once	NMF	Separate mixed notes
Too many weak false notes	Sparse NMF	Prefer a few strong notes
Tiny random detections	HMM	Make note activity smooth
Repeated notes merge together	Onset Detection	Find individual attacks
Harmonics look like notes	Ghost Filtering	Identify and remove overtones
Pitch jumps an octave	Median / Octave Fix	Correct likely pitch errors
Wrong random pitches	Key-Scale Filter	Use musical key as a constraint
Too many simultaneous notes	Polyphony Limit	Respect instrument capability
Impossible guitar fingering	Guitar Filter	Respect physical instrument limits
Voice has one pitch	pYIN	Track the fundamental frequency
Voice vibrato	Median Smoothing	Find underlying musical pitch

Math Isn't Enough

Music doesn't always follow mathematical rules



Integrating Music Theory

Problems with Pure DSP

- Overtones of C3 resemble C4, G4, E5
- Bass frequencies bleed across bins
- NMF cannot separate root from overtone
- Chords jump randomly with no logic
- No concept of *key* or *scale*

Example: A song in C Major should favour C, F, G, Am, Dm and Em — not random F#dim or Bb chords.

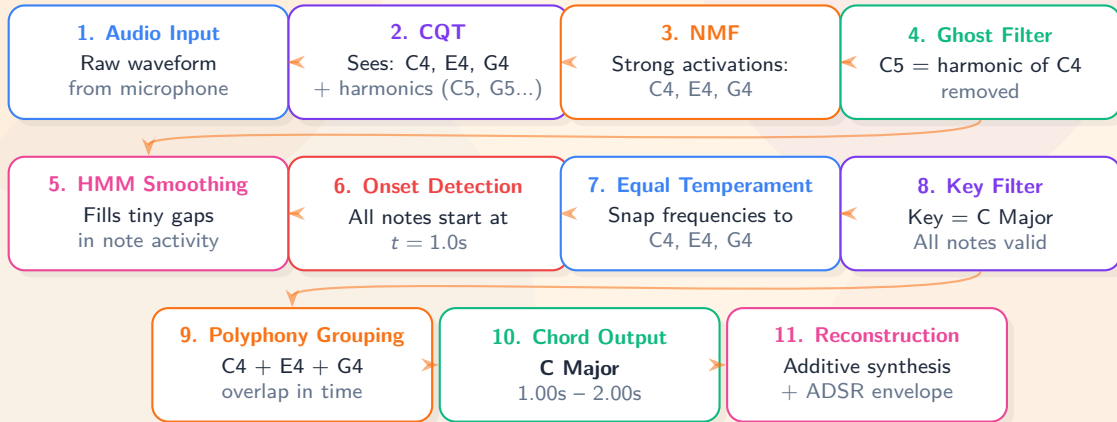
Our Music Theory Layer

- **Key Detection** — finds the tonal centre
- **Diatonic Filtering** — prioritises in-key chords
- **Bass Heuristics** — lowest note hints the root
- **Overtone Suppression** — removes phantom notes
- **HMM Smoothing** — stops illogical jumps
- **Scale-Aware Scoring** — musically weighted results



Complete Example — Detecting a C Major Chord

Input: A piano recording playing **C4 + E4 + G4** at $t = 1.0s$



Final Result: Original piano → Transcription (C4, E4, G4 = **C Major**) → Reconstructed synthesized piano for comparison.



Audio Lab — Effects Engine

Available Effects

- **Pitch Shifting** — change key instantly
- **Speed Control** — slow down to practise
- **3-Band EQ** — bass, mid, treble
- **Reverb & Delay** — spatial effects
- **Distortion** — guitar-style drive
- **Compressor** — dynamic control
- **Autotune** — live pitch correction
- **Record & Download** — save your mix

How It Works

Built with **Tone.js** and the **Web Audio API**. Every effect is chained in real time and processed entirely inside the browser — no server needed.

Autotune Engine

Detects vocal pitch using autocorrelation, snaps it to the nearest scale note, and applies smooth correction at 60 fps.



Conclusion

What We Achieved

- Translated raw audio into musical meaning using CQT and NMF
- Combined signal processing with real music theory
- Delivered a full practice toolkit in one place

Key Takeaway

We paired DSP with music theory, which produces results that are both technically accurate and musically sensible.

Crescendo



Thank You

Praggo Protim Datta

Roll: 2305060

Fahmid Hisham

Roll: 2305058

